

Task 8/8/2026

Varsha Sri Chukka 2303A52494

Prefix Problems:

[1480. Running Sum of 1d Array](#)

Given an array `nums`. We define a running sum of an array as `runningSum[i] = sum(nums[0]...nums[i])`.

Return the running sum of `nums`.

Code:

```
class Solution {
    public int[] runningSum(int[] nums) {
        int[] prefix = new int[nums.length];
        int sum=0;
        for(int i =0;i<nums.length;i++){
            sum+=nums[i];
            prefix[i]=sum;
        }
        return prefix;
    }
}
```

[303. Range Sum Query - Immutable](#)

Given an integer array `nums`, handle multiple queries of the following type:

1. Calculate the **sum** of the elements of `nums` between indices `left` and `right` **inclusive** where `left <= right`.

Implement the `NumArray` class:

- `NumArray(int[] nums)` Initializes the object with the integer array `nums`.
- `int sumRange(int left, int right)` Returns the **sum** of the elements of `nums` between indices `left` and `right` **inclusive** (i.e. `nums[left] + nums[left + 1] + ... + nums[right]`).

Code:

```
class NumArray {
    int[] prefix;
    public NumArray(int[] nums) {
        prefix = new int[nums.length];
        prefix[0]=nums[0];
        for(int i=1;i<nums.length;i++){
            prefix[i]=prefix[i-1]+nums[i];
        }
    }
}
```

```

    }

    public int sumRange(int left, int right) {
        if(left==0) return prefix[right];
        return prefix[right]-prefix[left-1];
    }
}

```

[1991. Find the Middle Index in Array](#)

Given a **0-indexed** integer array `nums`, find the **leftmost** `middleIndex` (i.e., the smallest amongst all the possible ones).

A `middleIndex` is an index where $\text{nums}[0] + \text{nums}[1] + \dots + \text{nums}[\text{middleIndex}-1] == \text{nums}[\text{middleIndex}+1] + \text{nums}[\text{middleIndex}+2] + \dots + \text{nums}[\text{nums.length}-1]$.

If `middleIndex == 0`, the left side sum is considered to be 0. Similarly, if `middleIndex == nums.length - 1`, the right side sum is considered to be 0.

Return *the **leftmost** `middleIndex` that satisfies the condition, or -1 if there is no such index.*

Code:

```

class Solution {
    public int findMiddleIndex(int[] nums) {
        int totalSum = 0;
        for(int x:nums)
            totalSum+=x;
        int leftSum = 0;
        for(int i =0;i<nums.length;i++){
            int rightSum=totalSum-leftSum-nums[i];
            if(leftSum==rightSum) return i;
            leftSum+=nums[i];
        }
        return -1;
    }
}

```

[724. Find Pivot Index](#)

Given an array of integers `nums`, calculate the **pivot index** of this array.

The **pivot index** is the index where the sum of all the numbers **strictly** to the left of the index is equal to the sum of all the numbers **strictly** to the index's right.

If the index is on the left edge of the array, then the left sum is 0 because there are no elements to the left. This also applies to the right edge of the array.

Return *the **leftmost pivot index**. If no such index exists, return -1.*

Code:

```
class Solution {
    public int pivotIndex(int[] nums) {
        int sum=0;
        for(int i:nums)
            sum+=i;
        int sum2=0;
        for(int i=0;i<nums.length;i++){
            if(sum-nums[i]==sum2) return i;
            else {sum-=nums[i];
                sum2+=nums[i];}
        }
        return -1;
    }
}
```

560. Subarray Sum Equals K

Given an array of integers nums and an integer k, return *the total number of subarrays whose sum equals to k*.

A subarray is a contiguous **non-empty** sequence of elements within an array.

Code:

```
class Solution {
    public int subarraySum(int[] nums, int k) {
        int[] prefix = new int[nums.length];
        prefix[0]=nums[0];
        for(int i =1;i<nums.length;i++){
            prefix[i]=prefix[i-1]+nums[i];
        }
        int c=0;
        for (int i = 0; i < nums.length; i++) {
            for (int j = i; j < nums.length; j++) {
                int sum;
                if (i == 0) {
                    sum = prefix[j];
                } else {
                    sum = prefix[j] - prefix[i - 1];
                }
                if (sum == k) {
                    c++;
                }
            }
        }

        return c;
    }
}
```

525. Contiguous Array

Given a binary array `nums`, return *the maximum length of a contiguous subarray with an equal number of 0 and 1*.

Code:

```
class Solution {
    public int findMaxLength(int[] nums) {
        int n = nums.length;
        for (int i = 0; i < n; i++) {
            if (nums[i] == 0) {
                nums[i] = -1;
            }
        }
        int[] prefix = new int[n];
        prefix[0] = nums[0];

        for (int i = 1; i < n; i++) {
            prefix[i] = prefix[i - 1] + nums[i];
        }

        int maxLen = 0;
        for (int i = 0; i < n; i++) {
            for (int j = i; j < n; j++) {
                int sum;
                if (i == 0) {
                    sum = prefix[j];
                } else {
                    sum = prefix[j] - prefix[i - 1];
                }

                if (sum == 0) {
                    maxLen = Math.max(maxLen, j - i + 1);
                }
            }
        }
        return maxLen;
    }
}
```

523. Continuous Subarray Sum

Given an integer array `nums` and an integer `k`, return true *if nums has a **good subarray*** or false *otherwise*.

A **good subarray** is a subarray where:

- its length is **at least two**, and
- the sum of the elements of the subarray is a multiple of `k`.

Note that:

- A **subarray** is a contiguous part of the array.
- An integer x is a multiple of k if there exists an integer n such that $x = n * k$. 0 is **always** a multiple of k .

Code:

```
class Solution {
    public boolean checkSubarraySum(int[] nums, int k) {
        int n = nums.length;
        int[] prefix = new int[n];
        prefix[0] = nums[0];

        for (int i = 1; i < n; i++) {
            prefix[i] = prefix[i - 1] + nums[i];
        }
        for (int i = 0; i < n; i++) {
            for (int j = i + 1; j < n; j++) {

                int sum;
                if (i == 0) {
                    sum = prefix[j];
                } else {
                    sum = prefix[j] - prefix[i - 1];
                }

                if (k == 0) {
                    if (sum == 0) return true;
                } else {
                    if (sum % k == 0) return true;
                }
            }
        }

        return false;
    }
}
```

Binary Search Problems:

[704. Binary Search](#)

Given an array of integers `nums` which is sorted in ascending order, and an integer `target`, write a function to search `target` in `nums`. If `target` exists, then return its index. Otherwise, return -1.

You must write an algorithm with $O(\log n)$ runtime complexity.

Code:

```
class Solution {
    public int search(int[] nums, int target) {
        int l=0,r=nums.length-1;
        while(l<=r){
            int mid = l+(r-l)/2;
            if(nums[mid]==target) return mid;
            else if(nums[mid]<target) l=mid+1;
            else r=mid-1;
        }
        return -1;
    }
}
```

35. Search Insert Position

Given a sorted array of distinct integers and a target value, return the index if the target is found. If not, return the index where it would be if it were inserted in order.

You must write an algorithm with $O(\log n)$ runtime complexity.

Code:

```
class Solution {
    public int searchInsert(int[] nums, int target) {
        int left=0;
        int right=nums.length;
        while(left<right){
            int mid =(left+right)/2;
            if(nums[mid]>=target)
                right=mid;
            else if(nums[mid]<target)
                left=mid+1;
        }
        return left;
    }
}
```

278. First Bad Version

You are a product manager and currently leading a team to develop a new product. Unfortunately, the latest version of your product fails the quality check. Since each version is developed based on the previous version, all the versions after a bad version are also bad.

Suppose you have n versions $[1, 2, \dots, n]$ and you want to find out the first bad one, which causes all the following ones to be bad.

You are given an API `bool isBadVersion(version)` which returns whether version is bad. Implement a function to find the first bad version. You should minimize the number of calls to the API.

Code:

```
/* The isBadVersion API is defined in the parent class VersionControl.
   boolean isBadVersion(int version); */

public class Solution extends VersionControl {
    public int firstBadVersion(int n) {
        int l=0;
        int r=n;
        while(l<n){
            int mid = l+(r-l)/2;
            if(isBadVersion(mid)){
                if(isBadVersion(mid-1))
                    r=mid-1;
                else
                    return mid;
            }
            else
                l=mid+1;
        }
        return n;
    }
}
```

69. Sqrt(x)

Given a non-negative integer x , return *the square root of x rounded down to the nearest integer*. The returned integer should be **non-negative** as well.

You **must not use** any built-in exponent function or operator.

- For example, do not use `pow(x, 0.5)` in c++ or `x ** 0.5` in python.

Code:

```
class Solution {
    public int mySqrt(int x) {
        if (x==0) return 0;
        long l =1,r=x;
        while(l<=r){
            long m = l+(r-l)/2;
            long sq = m*m;
            if(x==sq) return (int)m;
            else if(sq<x) l=m+1;
            else r=m-1;
        }
        return (int)r;
    }
}
```

```
}  
}
```

34. Find First and Last Position of Element in Sorted Array

Given an array of integers `nums` sorted in non-decreasing order, find the starting and ending position of a given target value.

If target is not found in the array, return `[-1, -1]`.

You must write an algorithm with $O(\log n)$ runtime complexity.

Code:

```
class Solution {  
    public int[] searchRange(int[] nums, int target) {  
        int[] arr = new int[2];  
        arr[0]=firstpos(nums,target);  
        arr[1]=lastpos(nums,target);  
        return arr;  
    }  
    int firstpos(int[] nums,int target){  
        int n = nums.length;  
        int l=0,h=n-1;  
        int res=-1;  
        while(l<=h){  
            int mid = l+(h-l)/2;  
            if(nums[mid]==target){  
                res = mid;  
                h=mid-1;  
            }  
            else if(nums[mid]<target)  
                l=mid+1;  
            else  
                h=mid-1;  
        }  
        return res;  
    }  
    int lastpos(int[] nums,int target){  
        int n = nums.length;  
        int l=0,h=n-1;  
        int res=-1;  
        while(l<=h){  
            int mid = l+(h-l)/2;  
            if(nums[mid]==target){  
                res = mid;  
                l=mid+1;  
            }  
            else if(nums[mid]<target)
```



```

        l=mid+1;
    else
        h=mid-1;
    }
    return res;
}
}

```

33. Search in Rotated Sorted Array

There is an integer array `nums` sorted in ascending order (with **distinct** values).

Prior to being passed to your function, `nums` is **possibly left rotated** at an unknown index `k` ($1 \leq k < \text{nums.length}$) such that the resulting array is `[nums[k], nums[k+1], ..., nums[n-1], nums[0], nums[1], ..., nums[k-1]]` (**0-indexed**). For example, `[0,1,2,4,5,6,7]` might be left rotated by 3 indices and become `[4,5,6,7,0,1,2]`.

Given the array `nums` **after** the possible rotation and an integer `target`, return *the index of target if it is in nums, or -1 if it is not in nums*.

You must write an algorithm with $O(\log n)$ runtime complexity.

Code:

```

class Solution {
    public int search(int[] nums, int target) {
        int n = nums.length;
        int l=0,h=n-1;
        while(l<=h){
            int mid = l+(h-l)/2;
            if(nums[mid]==target) return mid;
            if(nums[l]<=nums[mid]){
                if(nums[l]<=target&&target<=nums[mid])
                    h=mid-1;
                else
                    l=mid+1;
            }
            else{
                if(nums[mid]<target&& target<=nums[h])
                    l=mid+1;
                else
                    h=mid-1;
            }
        }
        return -1;
    }
}

```

162. Find Peak Element

A peak element is an element that is strictly greater than its neighbors.

Given a **0-indexed** integer array `nums`, find a peak element, and return its index. If the array contains multiple peaks, return the index to **any of the peaks**.

You may imagine that `nums[-1] = nums[n] = -∞`. In other words, an element is always considered to be strictly greater than a neighbor that is outside the array.

You must write an algorithm that runs in $O(\log n)$ time.

Code:

```
class Solution {
    public int findPeakElement(int[] nums) {
        int l = 0, h = nums.length - 1;

        while (l < h) {
            int mid = l + (h - l) / 2;

            if (nums[mid] > nums[mid + 1]) {
                h = mid;
            } else {
                l = mid + 1;
            }
        }

        return l;
    }
}
```

[74. Search a 2D Matrix](#)

You are given an $m \times n$ integer matrix `matrix` with the following two properties:

- Each row is sorted in non-decreasing order.
- The first integer of each row is greater than the last integer of the previous row.

Given an integer `target`, return `true` *if target is in matrix* or `false` *otherwise*.

You must write a solution in $O(\log(m * n))$ time complexity.

Code:

```
class Solution {
    public boolean searchMatrix(int[][] matrix, int target) {
        int m = matrix.length;
        int n = matrix[0].length;

        int l = 0, r = m * n - 1;
```

```

while (l <= r) {
    int mid = l + (r - l) / 2;
    int row = mid / n;
    int col = mid % n;

    int val = matrix[row][col];

    if (val == target) return true;
    else if (val < target) l = mid + 1;
    else r = mid - 1;
}

return false;
}
}

```

875. Koko Eating Bananas

Koko loves to eat bananas. There are n piles of bananas, the i^{th} pile has $\text{piles}[i]$ bananas. The guards have gone and will come back in h hours.

Koko can decide her bananas-per-hour eating speed of k . Each hour, she chooses some pile of bananas and eats k bananas from that pile. If the pile has less than k bananas, she eats all of them instead and will not eat any more bananas during this hour.

Koko likes to eat slowly but still wants to finish eating all the bananas before the guards return.

Return the minimum integer k such that she can eat all the bananas within h hours.

Code:

```

class Solution {
    public int minEatingSpeed(int[] piles, int h) {
        int low=1, high = piles[0];
        for(int ele : piles)
            if(ele>high)
                high=ele;
        int res = high;
        while(low<=high){
            int mid = low+(high-low)/2;
            if(isPossible(piles,h,mid)){
                res = mid;
                high = mid-1;
            }
            else
                low=mid+1;
        }
        return res;
    }
}

```

```
boolean isPossible(int[] piles,int h,int mid){  
    long hrs=0;  
    for(int ele : piles)  
        hrs+=(ele+mid-1)/mid;  
    return hrs<=h;  
}  
}
```